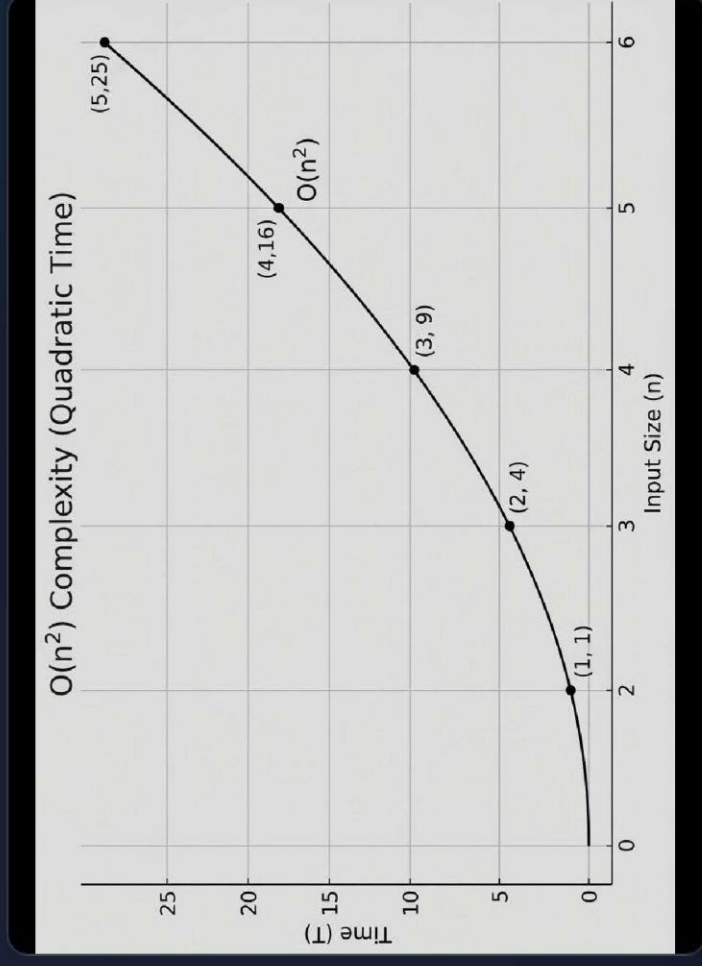


The Computational Bottleneck

Consider a simulation with **N** particles. In a naive approach, checking for collisions requires comparing every particle against every other particle.

$$\text{Comparisons} = \frac{N(N-1)}{2} \approx O(N^2)$$

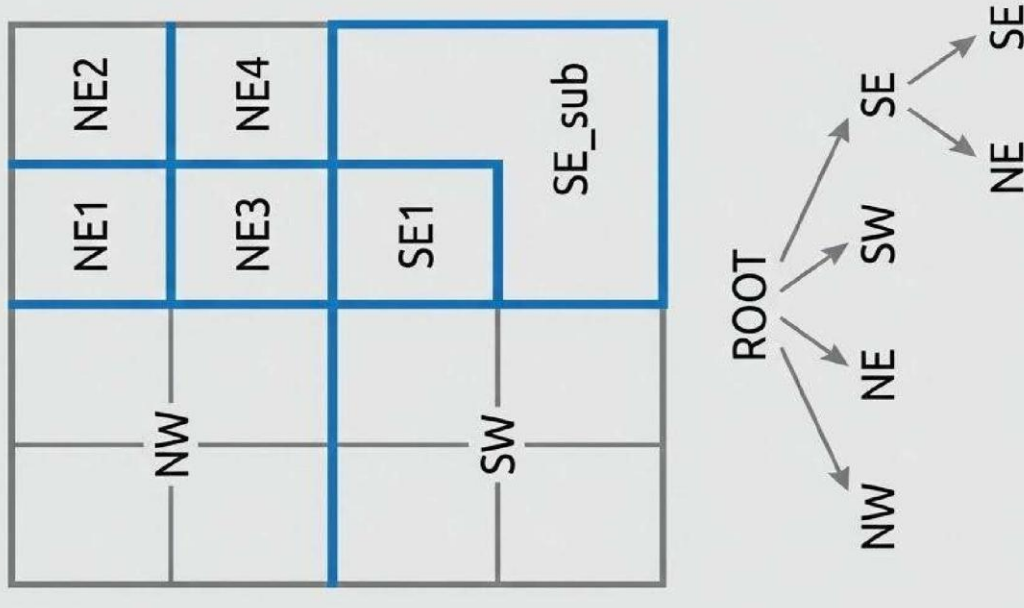


Spatial Partitioning

The Quadtree solves this by recursively decomposing the 2D space into four quadrants. This transforms the problem from a flat list check to a hierarchical tree traversal.

Instead of checking $O(N)$ entities, we check only those in the same local region, drastically reducing the search space.

QUADTREE MATRIX REPRESENTATION



Formal Definition

Let S be a square region in $\mathbb{R}^2$. A Quadtree node $Q(S)$ is defined recursively:

- **Leaf Node:** If $\text{capacity}(S) < k$, $Q(S)$ is a leaf containing points.
- **Internal Node:** If $\text{capacity}(S) \geq k$, S is partitioned into four disjoint quadrants:

$$Q(S) = \{ S_{NW}, S_{NE}, S_{SW}, S_{SE} \}$$

Where:

$$S = S_{NW} \cup S_{NE} \cup S_{SW} \cup S_{SE}$$

And $S_i \cap S_j = \emptyset$ for $i \neq j$.

```
Point{
    t x, y;
```

```
Node{
```

```
    int pos;
    t data;
```

```
Quad{
```

```
    int topLeft, botRight;
```

```
    tde bucket[4];
```

```
    t count;
```

```
    t*tl, *tr, *br, *bl;
```

```
    bool divided;
```

```
    createQuad(Point tl, Point br)
```

```
    Quad *q = new Quad;
```

```
    q->topLeft = tl;
```

```
    q->botRight = br;
```

```
    q->count = 0;
```

```
    q->tl = q->tr = q->bl = q->br = NULL;
```

```
    q->divided = false;
```

```
    return q;
```

80si0S uow

Utility Functions:

```
bool inBoundary(Quad *q, Point p)
{
    bool x_in = p.x >= q->topLeft.x && p.x <= q->botRight.x;
    bool y_in = p.y >= q->botRight.y && p.y <= q->topLeft.y;

    return x_in && y_in;
}

void subdivide(Quad *q)
{
    int midX = (q->topLeft.x + q->botRight.x) / 2;
    int midY = (q->topLeft.y + q->botRight.y) / 2;

    cout << 3 << "\n0";
    q->tl = createQuad(q->topLeft, Point{midX, midY});
    q->tr = createQuad(Point{midX, q->topLeft.y}, Point{q->botRight.x, midY});
    q->bl = createQuad(Point{q->topLeft.x, midY}, Point{midX, q->botRight.y});
    q->br = createQuad(Point{midX, midY}, q->botRight);

    q->divided = true;
}
```

```

Node *searchNode(Quad *q, Point p)
{
    if (!inBoundary(q, p))
        return NULL;

    for (int i = 0; i < q->count; i++)
    {
        if (q->bucket[i].pos.x == p.x && q->bucket[i].pos.y == p.y)
            return &q->bucket[i];
    }

    if (!q->divided)
        return NULL;

    int midX = (q->topLeft.x + q->botRight.x) / 2;
    int midY = (q->topLeft.y + q->botRight.y) / 2;

    if (p.x <= midX)
    {
        if (p.y <= midY)
            return searchNode(q->tl, p);
        else
            return searchNode(q->bl, p);
    }
    else
    {
        if (p.y <= midY)
            return searchNode(q->tr, p);
        else
            return searchNode(q->br, p);
    }
}

```


Range Query:

```
bool intersects(Quad *q, Point rangeTL, Point rangeBR)
{
    if (q->botRight.x < rangeTL.x || q->topLeft.x > rangeBR.x)
        return false;

    if (q->botRight.y > rangeTL.y || q->topLeft.y < rangeBR.y)
        return false;
    return true;
}

void rangeQuery(Quad *q, Point rangeTL, Point rangeBR, vector<Node> &result)
{
    if (q == NULL || !intersects(q, rangeTL, rangeBR))
        return;

    for (int i = 0; i < q->count; i++)
    {
        Node &node = q->bucket[i];

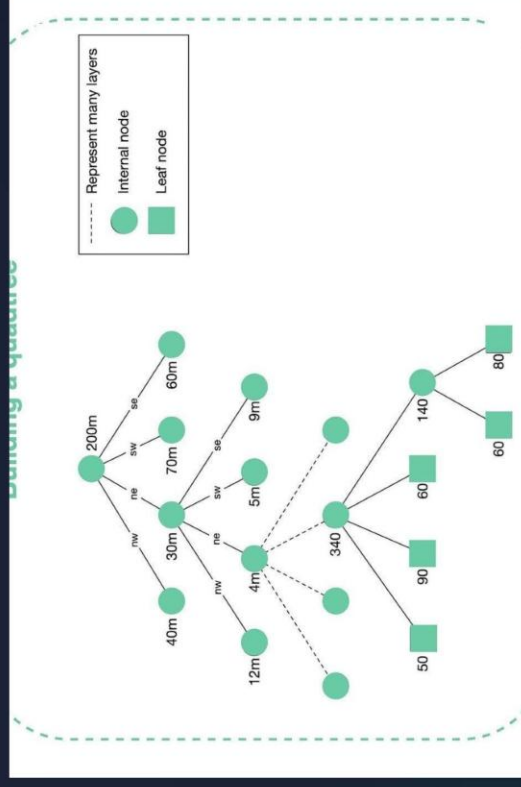
        if (node.pos.x >= rangeTL.x && node.pos.x <= rangeBR.x &&
            node.pos.y >= rangeBR.y && node.pos.y <= rangeTL.y)
        {
            result.push_back(node);
        }
    }

    if (q->divided)
    {
        rangeQuery(q->tl, rangeTL, rangeBR, result);
        rangeQuery(q->tr, rangeTL, rangeBR, result);
        rangeQuery(q->bl, rangeTL, rangeBR, result);
        rangeQuery(q->br, rangeTL, rangeBR, result);
    }
}
```

Tree Structure vs. Spatial Grid

The Spatial View

Geometrically, we see squares subdividing wherever point density is high. This is an adaptive grid that provides high resolution only where needed.



The Graph View

Structurally, this forms a quaternary tree. The root represents the entire world. Each level down represents a "zoom in" by a factor of 2.

- **Root:** Global Bounds
- **Degree:** Exactly 4 (Internal Nodes)
- **Leaves:** Data buckets

Type of points distribution and time complexity

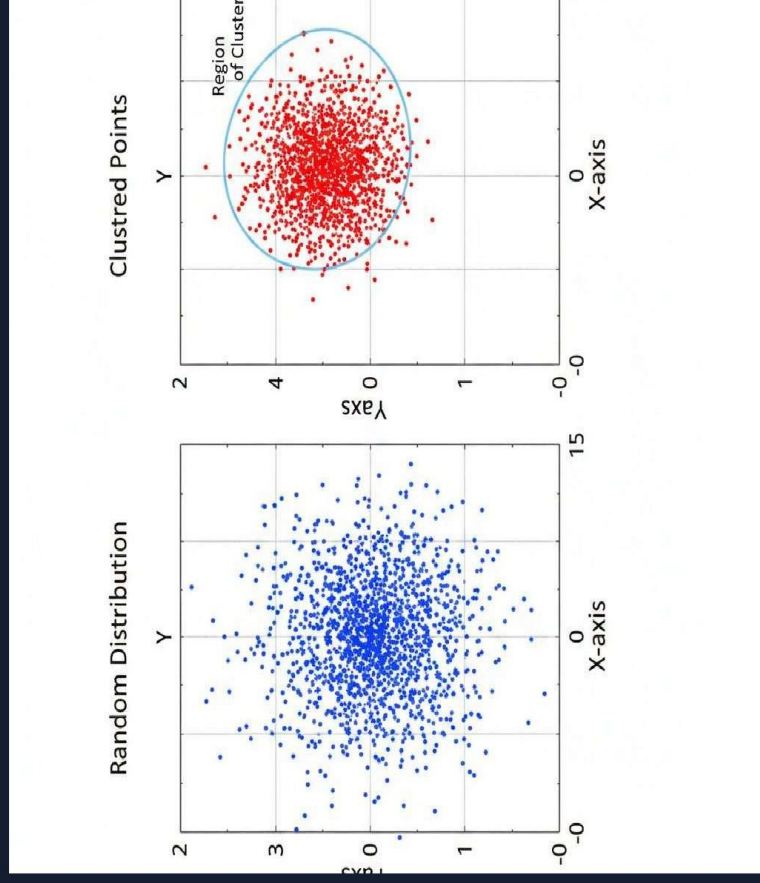
n -> Total number of points

k -> number of points in the search query range

$D_{\max}$ varies according to point distribution.

In case of uniform distribution $D_{\max} = O(\log n)$

In case of clustered $D_{\max} = O(n)$



A -> Random distribution

Insertion - $O(D_{\max})$

Search - $O(D_{\max})$

Search query - $O(D_{\max} + k)$

B -> Clustered points

Insertion - $O(D_{\max})$

Search - $O(D_{\max})$

Search query - $O(D_{\max} + k)$

Complexity Comparison:

For Clustered data: $O(n^2)$

For Uniform data: $O(n \log n)$

